HQC — O que mudou entre o Golden e as versões modificadas

Explicação detalhada dos três experimentos de alteração controlada em RTL

> **Escopo e validação:** as mudanças foram realizadas em cópias de três módulos; o golden original foi preservado. Foram executados testes em modelos Python. **Os arquivos Verilog modificados ainda não foram simulados com Icarus Verilog** neste ambiente. A análise abaixo descreve os experimentos, não uma quebra demonstrada do HQC.

1. HT-GF — Corrupção seletiva na multiplicação

Módulo: `hardware/encap/gf_mul.v`

Esse módulo implementa a multiplicação no corpo finito GF(2^8), utilizado nas operações de codificação Reed–Solomon.

Código Golden (original)

```
always @(posedge clk)
begin
    out_reg <= mul_7;
    done_reg_2 <= done_reg_1;
end
```

O circuito armazena no registrador de saída o resultado da multiplicação em GF(2^8).

Código modificado

```
wire [7:0] mul_selected =
    ((in_1_reg == 8'hA5) && (in_2_reg == 8'h5A))
    ? (mul_7 ^ 8'h01)
    : mul_7;
```

A linha de atualização do registrador passa de:

```
- out_reg <= mul_7;
+ out_reg <= mul_selected;
```

Quando os operandos registrados são **0xA5** e **0x5A**, o resultado sofre um XOR com `0x01`, invertendo o bit menos significativo. Nos demais casos, `mul_selected` mantém o valor original.

Situação	Golden	Modificado
Entrada A	0xA5	0xA5
Entrada B	0x5A	0x5A
Resultado	**0xA6**	**0xA7**

Situação	Golden	Modificado
Sinal de conclusão	Inalterado	Inalterado

Trigger: par específico de operandos. **Payload:** inversão do bit 0 no resultado. A lógica de conclusão não foi alterada.

Teste de referência Python

Foram avaliadas todas as **65.536 combinações** de operandos de oito bits: 65.535 permaneceram iguais e **1** produziu divergência. Isso ilustra uma condição de ativação rara sob amostragem uniforme, mas não prova, por si só, furtividade.

> **Detalhe de escopo:** a modificação afeta o caminho registrado `REG_OUT=1`. O caminho alternativo `REG_OUT=0` não foi modificado.

2. HT-CTRL — Conclusão sem gravação do último byte

Módulo: `hardware/decap/hqc_rmdecod_ctrl.v`

Esse controlador coordena a escrita na RAM dos bytes recuperados pela decodificação Reed–Muller.

Código Golden (original)

```
assign ram_dout_o      = peak_dout_i;
assign ram_dout_wr_o   = peak_valid_i;
assign ram_dout_addr_o = cnt_out;
```

Quando `peak_valid_i` está ativo, o byte decodificado pode ser gravado na RAM.

Código modificado

```
- assign ram_dout_wr_o = peak_valid_i;
+ assign ram_dout_wr_o = peak_valid_i &
+     ~((cnt_out == N1-1) &&
+       (peak_dout_i == 8'hA5));
```

Na configuração de segurança de 128 bits, o valor padrão de `N1` é 46. Assim, a condição adicional desativa a escrita quando o contador está no índice final (**45**) e o byte decodificado é **0xA5**.

Condição	Golden	Modificado
`cnt_out == 45`	Sim	Sim
`peak_dout_i == 0xA5`	Sim	Sim
`peak_valid_i == 1`	Sim	Sim
Escrita na RAM	**Sim**	**Não**

Condição	Golden	Modificado
Sinalização de conclusão	Sim	Sim

A lógica de conclusão permaneceu baseada na condição original:

```
assign last_dout =
    peak_valid_i & cnt_out==(N1-1);
```

Portanto, a escrita do último byte é suprimida, mas a condição de conclusão pode continuar satisfeita. Isso gera uma **inconsistência entre o estado da memória e a sinalização de término**.

Trigger: último byte igual a 0xA5. **Payload:** inibição da gravação correspondente.

Teste de referência Python

Com 256 possibilidades para o último byte, 255 mantiveram o comportamento de escrita e **1** desabilitou a escrita. O estado de conclusão modelado permaneceu inalterado.

> Essa diferença demonstra uma falha de integridade no controle local. **Não demonstra, sem simulação completa, alteração do segredo compartilhado ou comprometimento da segurança criptográfica.**

3. HT-PEAK — Alteração do desempate Reed–Muller

Módulo: `hardware/decap/hqc_rmdecod_findpeaks.v`

O módulo seleciona o pico mais relevante durante a decodificação. Se os valores absolutos de dois picos são iguais, a posição funciona como critério de desempate.

Código Golden (original)

```
assign check_abs = (peak_abs1 > peak_abs0);
assign check_equ = (peak_abs1 == peak_abs0);
assign check_pos = (peak_pos1 > peak_pos0);
```

Código modificado

```
- assign check_pos = (peak_pos1 > peak_pos0);
+ assign check_pos = (peak_pos1 < peak_pos0);
```

Somente o operador de desempate foi invertido. A comparação das magnitudes continua igual.

Exemplo de efeito

Propriedade	Pico 0	Pico 1
Magnitude	7	7
Posição	20	21

Seleção	Golden	Modificado
Posição escolhida	**21**	**20**
Saída correspondente	**0x95**	**0x94**

Quando uma magnitude é estritamente maior, o critério de desempate não interfere nessa escolha. O desvio ocorre nas condições de empate em que a orientação da posição altera a seleção.

Teste de referência Python

Foram consideradas **961 combinações** de magnitudes (31 × 31), com posições fixas: **930** sem divergência e **31** com divergência, correspondentes aos empates.

Além dessa mudança funcional, foram renomeados os módulos `hqc_rmdecod_findpeaks_mutant` e `findpeaks_core_mutant`, bem como suas duas instanciações, para permitir a coexistência de golden e variante no mesmo testbench. Essas renomeações, isoladamente, não constituem payload.

4. Resumo — o que foi alterado

Experimento	Mudança principal	Trigger	Efeito local
HT-GF	Seletor com XOR 0x01	Operandos 0xA5 / 0x5A	Corrupção de 1 bit
HT-CTRL	Condição adicional na habilitação de escrita	Último byte 0xA5	Supressão da escrita
HT-PEAK	Troca do operador > por <	Empate de magnitudes	Escolha de outro pico

Foram modificadas apenas cópias dos três módulos mencionados. Não houve alteração do sampler, SHAKE ou verificação Fujisaki–Okamoto. As variantes não foram integradas nem testadas em uma execução completa de KeyGen, Encaps e Decaps.

5. Significado para um artigo científico

Os três cenários representam classes distintas de falha em RTL: **HT-GF** altera dados em uma unidade aritmética; **HT-CTRL** modifica o controle de gravação e a consistência de conclusão; e **HT-PEAK** altera uma decisão algorítmica na decodificação.

O valor para a pesquisa com LLMs é observar se os modelos identificam as mudanças, localizam seus efeitos e propõem testes capazes de exercitá-las. É necessário distinguir os níveis de evidência:

- **Confirmado:** divergências em modelos de referência Python nos cenários descritos.
- **A confirmar:** execução dos testbenches Verilog e comportamento ciclo a ciclo no Icarus Verilog.
- **Não demonstrado:** impacto em decapsulação HQC completa, quebra criptográfica, vazamento de chave ou ataque físico.

> **Próxima etapa:** integrar HT-CTRL e HT-PEAK no HQC completo e comparar golden e variante em uma decapsulação ponta a ponta. Só então avaliar o impacto criptográfico.